



《大规模语言模型：从理论到实践》

第5章 有监督微调

张奇 桂韬 郑锐 黄萱菁

复旦大学



有监督微调（Supervised Finetuning, SFT） 又称指令微调（Instruction Tuning），是指在已经训练好的语言模型的基础上，通过使用有标注的特定任务数据进行进一步的微调，使模型具备遵循指令的能力。

经过海量数据预训练后的语言模型虽然具备了大量的“知识”，但是由于其训练时的目标仅是进行下一个词的预测，因此不能够理解并遵循人类自然语言形式的指令。为了使模型具有理解并响应人类指令的能力，还需要使用指令数据对其进行微调。

如何构造指令数据，如何高效低成本地进行指令微调训练，以及如何在语言模型基础上进一步扩大上下文等问题，是大语言模型在有监督微调阶段的核心。

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考



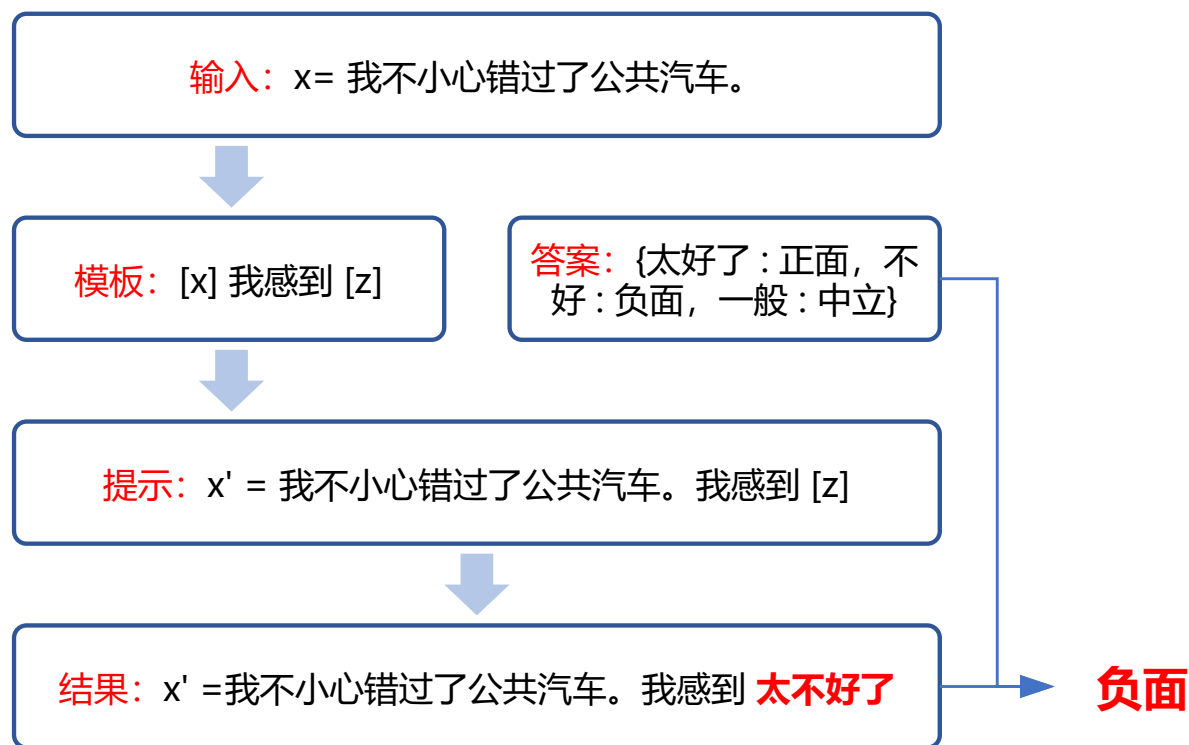
在指令微调大模型的方法之前，如何高效地使用预训练好的基座语言模型是学术界和工业界关注的热点。提示学习逐渐成为大语言模式使用的新范式。

与传统的微调方法不同，提示学习基于语言模型方法来适应下游各种任务，**通常不需要参数更新**。然而，由于所涉及的检索和推断方法多种多样，不同模型、数据集和任务都有不同的预处理要求，提示学习的实施十分复杂。

本节将介绍提示学习的大致框架，以及基于提示学习演化而来的语境学习方法。



提示学习 (Prompt-based Learning) 不同于传统的监督学习，它直接利用了有大量原始文本上进行预训练的语言模型，并通过定义一个新的提示函数，使该模型能够执行小样本甚至零样本学习，以适应仅有少量标注或没有标注数据的新场景。



原始输入 x 经过一个模板，被修改成一个带有一些未填充槽的文本提示 x' ，然后将这段提示输入语言模型，语言模型即以概率的方式填充模板中待填充的信息，然后根据模型的输出导出最终的预测标签 y



使用提示学习完成预测的整个过程可以描述为三个阶段：提示添加、答案搜索、答案映射。

(1) 提示添加：在这一步骤中，需要借助特定的模板，将原始的文本和额外添加的提示拼接起来，一并输入到语言模型中。例如，在情感分类任务中，根据任务的特性，可以构建如下含有两个插槽的模板：

“[X] 我感到[Z]”

其中，[X] 插槽中填入待分类的原始句子，[Z] 插槽中为需要语言模型生成的答案。假如原始文本

$x =$ “我不小心错过了公共汽车。”

通过此模板，整段提示将被拼接成

$x' =$ “我不小心错过了公共汽车。我感到[Z]”



(2) 答案搜索：将构建好的提示整体输入语言模型后，需要找出语言模型对[Z]处预测得分最高的文本 $\hat{z}$ 。根据任务特性，**可以事先定义预测结果z 的答案空间**为Z。在简单的生成任务中，答案空间可以涵盖整个语言，而在一些分类任务中，答案空间可以是一些限定的词语，例如

$$Z = \{ \text{“太好了”}, \text{“好”}, \text{“一般”}, \text{“不好”}, \text{“糟糕”} \}$$

这些词语可以分别映射到该任务的最终标签上。将给定提示为 x' 而模型输出为 z 的过程记录为函数 $f_{\text{fill}}(x', z)$ ，对于每个答案空间中的候选答案，分别计算模型输出它的概率，从而找到模型对[Z] 插槽预测得分最高的输出：

$$\hat{z} = \text{search}_{z \in Z} P(f_{\text{fill}}(x', z); \theta)$$



(3) 答案映射：得到的模型输出 $\hat{z}$ 并不一定就是最终的标签。在分类任务中，还需要将模型的输出与最终的标签做映射。而这些映射规则是人为制定的，例如，
“太好了” “好” 映射为 “正面” 标签，将 “不好” “糟糕” 映射为 “负面” 标签，
将 “一般” 映射为 “中立” 标签。

$$\begin{cases} \text{若 } \hat{z} \in \{\text{“太好了”}, \text{“好”}\} & \hat{y} = \text{“正面”} \\ \text{若 } \hat{z} \in \{\text{“不好”}, \text{“糟糕”}\} & \hat{y} = \text{“负面”} \\ \text{若 } \hat{z} \in \{\text{“一般”}\} & \hat{y} = \text{“中立”} \end{cases}$$



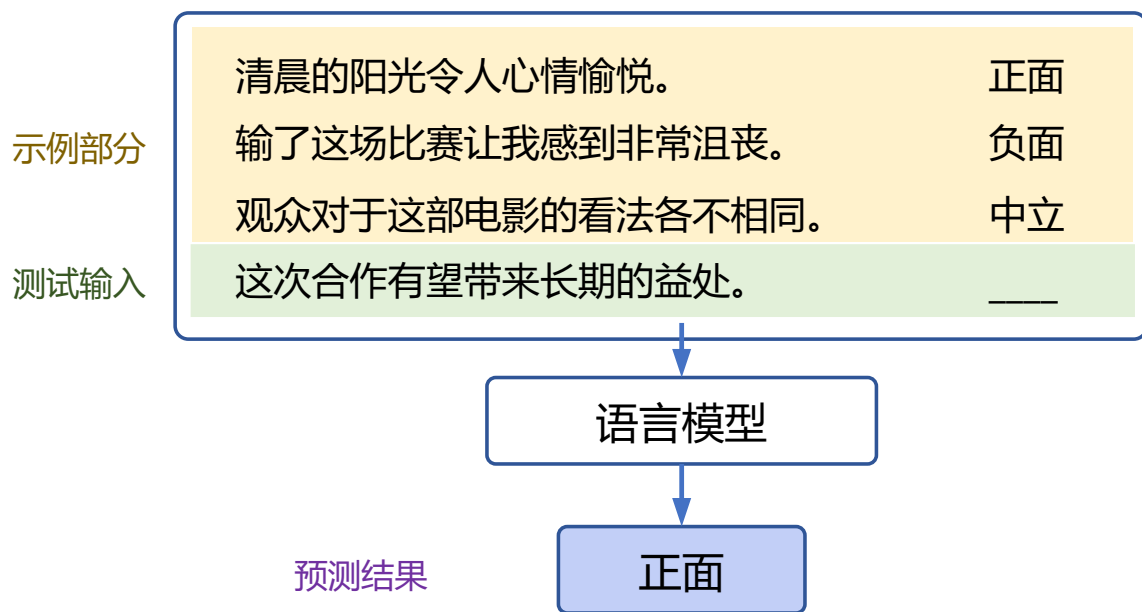
此外，由于提示构建的目的是找到一种方法，使语言模型有效地执行任务，并不需要将提示限制为人类可解释的自然语言。因此，也有人研究连续提示的方法，即**软提示 (Soft Prompt)**，其直接在模型的嵌入空间中执行提示。具体来说，连续提示删除了两个约束：

- (1) 不再要求提示词是自然语言。
- (2) 模板不再受语言模型自身参数的限制。相反，模板有自己的参数，可以根据下游任务的训练数据进行调整。

提示学习方法易于理解且效果显著，提示工程、答案工程、多提示学习方法、基于提示的训练策略等已经成为从提示学习衍生出的新的研究方向



语境学习 (In-context Learning, ICL)，也称上下文学习，其概念随着GPT-3的诞生而被提出。语境学习是指模型**可以从上下文中的几个例子中学习**：向模型输入特定任务的一些具体例子（也称示例〔Demonstration〕）以及要测试的样例，模型可以根据给定的示例续写测试样例的答案。



向模型中输入一些带有情感极性的句子、每条句子相应的标签、以及待测试的句子，模型可以自然地续写出它的情感极性为“Positive”。

语境学习可以看作是提示学习的一个子类，其中示例是提示的一部分。语境学习的关键思想是从类比中学习，整个过程并不需要对模型进行参数更新，仅执行前向的推理。大语言模型可以通过语境学习执行许多复杂的推理任务。



语境学习作为大语言模型时代的一种新的范式，具有许多独特的优势。

- 首先，其示例是用自然语言编写的，这提供了一个可解释的界面来与大语言模型进行交互。
- 其次，不同于以往的监督训练，语境学习本身无须参数更新，这可以大大降低使大语言模型适应新任务的计算成本。

语境学习作为一种新兴的方法，其作用机制仍有待深入研究。文献[147] 指出，语境学习中示例的标签正确性（即输入和输出的具体对应关系）并不是使其行之有效的关键因素，并认为起到更重要作用的是输入和样本配对的格式、输入和输出分布等。此外，语境学习的性能对特定设置很敏感，包括提示模板、上下文内示例的选择及示例的顺序。如何通过语境学习方法更好地激活大语言模型已有的知识成为一个新的研究方向。

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考



随着更多长文本建模需求的出现，多轮对话、长文档摘要等任务在实际应用中越来越多，这些任务需要模型能够更好地处理超出常规上下文窗口大小的文本内容。尽管当前的大语言模型在处理短文本方面表现出色，但在支持长文本建模方面仍存在一些挑战，这些挑战包括预定义的上下文窗口大小限制等。

以MetaAI 在2023 年2 月开源的LLaMA 模型[37] 为例，其规定输入文本的词元数量不得超过2048个。这会限制模型对长文本的理解和表达能力。当涉及长时间对话或长文档摘要时，传统的上下文窗口大小可能无法捕捉到全局语境，从而导致信息丢失或模糊的建模结果。



为了更好地满足长文本需求，有必要探索如何扩展现有的大语言模型，使其能够有效地处理更大范围的上下文信息。具体来说，主要有以下方法来扩展语言模型的长文本建模能力。

- **增加上下文窗口的微调：**采用直接的方式，即通过使用一个更大的上下文窗口来微调现有的预训练Transformer，以适应长文本建模需求。
- **位置编码：**改进的位置编码，如ALiBi[69]、LeX[157] 等能够实现一定程度上的长度外推。这意味着它们可以在小的上下文窗口上进行训练，在大的上下文窗口上进行推理。
- **插值法：**将超出上下文窗口的位置编码通过插值法压缩到预训练的上下文窗口中。

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考



因为指令数据的质量会直接影响有监督微调的最终效果，所以指令数据的构建应是一个非常精细的过程。从获得来源上看，构建指令数据的方法可以分为手动构建指令和利用大语言模型的生成能力自动构建指令两种。



手动构建指令的方法比较直观，可以在网上收集大量的问答数据再人为加以筛选过滤，或者通过标注者直接手动编写提示与相应的回答。虽然这是一个比较耗费人力的过程，但其优势在于可以很好地把控指令数据的标注过程，并对整体质量进行很好的控制。

指令数据的质量和多样性通常被认为是衡量指令数据的两个最重要的维度。LIMA[47] 研究在一定程度上说明了高质量且多样的指令数据可以“以少胜多” (Less is More)。本节就以LIMA为例，详细介绍手动构建指令的过程及需要注意的细节。



为了保证指令数据的多样性，**LIMA 从多个来源采样收集指令数据**，包括高质量网络问答社区、Super-Natural Instructions[160] 指令集，以及大量的标注者手动编写的提示与回答。

- 网络问答社区包含多个子版块，涵盖了不同的主题。
- Super-Natural Instructions 指令集也包含了多种多样的生成式任务。
- 标注者各自编写的提示与回答具有天然的多样性，因此指令数据的多样性得到了很好的保障。



LIMA 做了大量的工作来保证指令数据的质量。

首先，指令数据来源的可靠已经在一定程度上保证了它的质量。

其次，LIMA 额外制定了一些规则进一步提高其质量。

- 例如，对社区指令数据采样时选择排名靠前的优质回答，将所有回答统一成AI 助手的风格，删除过长或过短的回答，删除以第一人称开头的回答，删除包含链接的回答，标注者精心手动编写回答，等等。

LIMA 的实验结果说明指令数据的质量和多样性是影响有监督微调过程的关键因素。LIMA仅使用1 000 条这样的指令数据，就可以媲美甚至超过指令数据是其几十倍的同等参数规模的其他模型。



LIMA 在训练时所使用的指令数据示例如下所示：

prompt:

Tell me an interesting fact about geography.

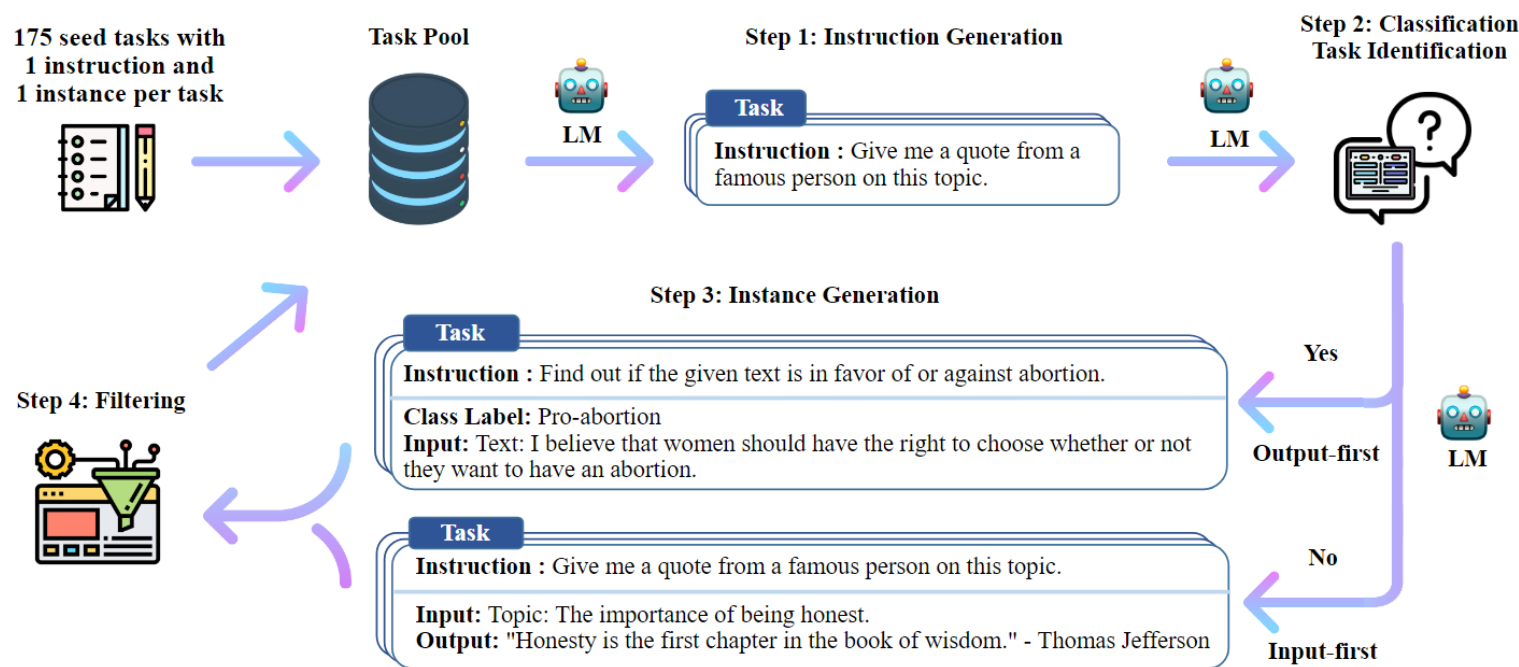
response:

Here are some interesting facts about geography:

- Bhutan is one of the most intriguing places in the world. The Buddhist kingdom is the only
↳ carbon-negative country on Earth, with strict environmental laws and cultural norms that
↳ preserve its stunning natural landscape. For similar reasons, Bhutan was one of the last
↳ countries to introduce television in 1999, and has no traffic lights to this day.
- There are currently few territories on Earth that are unclaimed by any sovereign state (terra
↳ nullius). One example is Bir Tawil, along the border between Egypt and Sudan, which was
↳ created by a discrepancy between borders drawn in 1899 and 1902. The territory remains
↳ unclaimed by either country, because it would entail forfeiting the much larger Halaib
↳ Triangle, which is claimed by both countries. Bir Tawil has no settled population, but the
↳ land is used by Bedouins who roam the area.
- The antipode of any spot on Earth [...]



手动构建指令数据代价高昂，需要大量的人力投入。因此，一些研究尝试寻找更高效的替代方法。具有代表性的工作如Self-Instruct[161]，利用大语言模型的生成能力自动构建指令。Self-instruct 数据生成过程是一个迭代引导算法。如图5.7所示，它包含以下四个步骤。





步骤1：生成任务指令

手动构建一个包含175 个任务的小型指令数据集，称为种子指令集，用于初始化指令池。然后让模型以自举（Bootstrapping）的方式，利用指令池，生成新任务的指令：每次从指令池中采样8 条任务指令（其中6 条来自人工编写的种子指令，2 条是模型迭代生成的），将其拼接为上下文示例，引导预训练语言模型GPT-3 生成更多的新任务的指令，直到模型自己停止生成，或达到模型长度限制，或是在单步中生成了过多示例（例如当出现了 “Task 16” 时）。



步骤2：确定指令是否代表分类任务

由于后续对于分类任务和非分类任务有两种不同的处理方法，因此需要在本步骤对指令是否、为分类任务进行判断，同样是利用拼接几个上下文示例的方法让模型自动判断任务类型是否是分类。



步骤3：生成任务输入和输出

通过步骤1，语言模型已经生成了面向新任务的指令，然而指令数据中还没有相应的输入和输出。本步骤将为此前生成的指令生成输入和输出，让指令数据变得完整。与之前的步骤相同，本步骤同样使用语境学习，使用来自其他任务的“指令”“输入”“输出”上下文示例做提示，预训练模型就可以为新任务生成输入-输出对。

针对不同的任务类别，分别使用“输入优先”或“输出优先”方法：对于非分类任务，使用输入优先的方法，先根据任务产生输入，然后根据任务指令和输入，生成输出；而对于分类任务，为了避免模型过多地生成某些特定类别的输入（而忽略其他的类别），使用输出优先的方法，即先产生所有可能的输出标签，再根据任务指令和输出，补充相应的输入。



步骤3：生成任务输入和输出

“输入优先”提示模板如下所示：

```
Come up with examples for the following tasks. Try to generate multiple examples when possible. If  
↔ the task doesn't require additional input, you can generate the output directly.
```

```
Task: Sort the given list ascendingly.
```

```
Example 1
```

```
List: [10, 92, 2, 5, -4, 92, 5, 101]
```

```
Output: [-4, 2, 5, 5, 10, 92, 92, 101]
```

```
Example 2
```

```
List: [9.99, 10, -5, -1000, 5e6, 999]
```

```
Output: [-1000, -5, 9.99, 10, 999, 5e6]
```

```
Task: Converting 85 F to Celsius.
```

```
Output: 85°F = 29.44°C
```

```
Task: {Instruction for the target task}
```



步骤3：生成任务输入和输出

“输出优先”提示模板如下所示：

```
Given the classification task definition and the class labels, generate an input that corresponds  
↪ to each of the class labels. If the task doesn't require input, just generate the correct  
↪ class label.
```

```
Task: Classify the sentiment of the sentence into positive, negative, or mixed.
```

```
Class label: mixed
```

```
Sentence: I enjoy the flavor of the restaurant but their service is too slow.
```

```
Class label: Positive
```

```
Sentence: I had a great day today. The weather was beautiful and I spent time with friends.
```

```
Class label: Negative
```

```
Sentence: I was really disappointed by the latest superhero movie. I would not recommend it.
```

```
Task: Tell me if the following email is a promotion email or not.
```

```
Class label: Promotion
```

```
Email: Check out our amazing new sale! We've got discounts on all of your favorite products.
```

```
Class label: Not Promotion
```

```
Email: We hope you are doing well. Let us know if you need any help.
```

```
Task: {Instruction for the target task}
```



步骤4：过滤低质量数据

为了保证数据的多样性，在将新生成的指令数据加入指令池之前，需要先衡量它和池中已有指令数据的相似度，只有当它和池中任何一条指令数据的ROUGE-L 相似度都低于0.7 时，才可能将其加入指令池。为保证数据的质量，还制定了一系列的启发式规则进行筛选：删除包含某些关键词（如“图片”）的指令数据、重复的指令数据、过长或过短的数据等。



指令数据集对于有监督微调非常重要，无论手工还是自动构建都需要花费一定的时间和成本。目前已经有一些开源指令数据集，本节将选择一些常用的指令数据集进行介绍。开源指令数据集按照指令任务的类型划分，可以分为传统NLP 任务指令和通用对话指令两类。表5.1 给出了部分开源指令数据集的汇总信息

表 5.1 部分开源指令数据集

指令数据集名称	指令数据集大小	语言	构建方式	指令类型
Super-Natural Instructions	500 万	多语言	手动构建	NLP 任务指令
Flan2021	44 万	英语	手动构建	NLP 任务指令
pCLUE	120 万	中文	手动构建	NLP 任务指令
OpenAssistant Conversations	16.1 万	多语言	手动构建	通用对话指令
Dolly	1.5 万	英语	手动构建	通用对话指令
LIMA	1 000	英语	手动构建	通用对话指令
Self-Instruct	5.2 万	英语	自动构建	通用对话指令
Alpaca_data	5.2 万	英语	自动构建	通用对话指令
BELLE	150 万	中文	自动构建	通用对话指令

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考



ChatGPT 整体的训练过程复杂，虽然基于DeepSpeed 可以通过单机多卡、多机多卡、流水线并行等操作来训练和微调大语言模型，但是没有端到端的基于人类反馈机制的强化学习的规模化系统，仍然会造成训练类ChatGPT 系统非常困难。

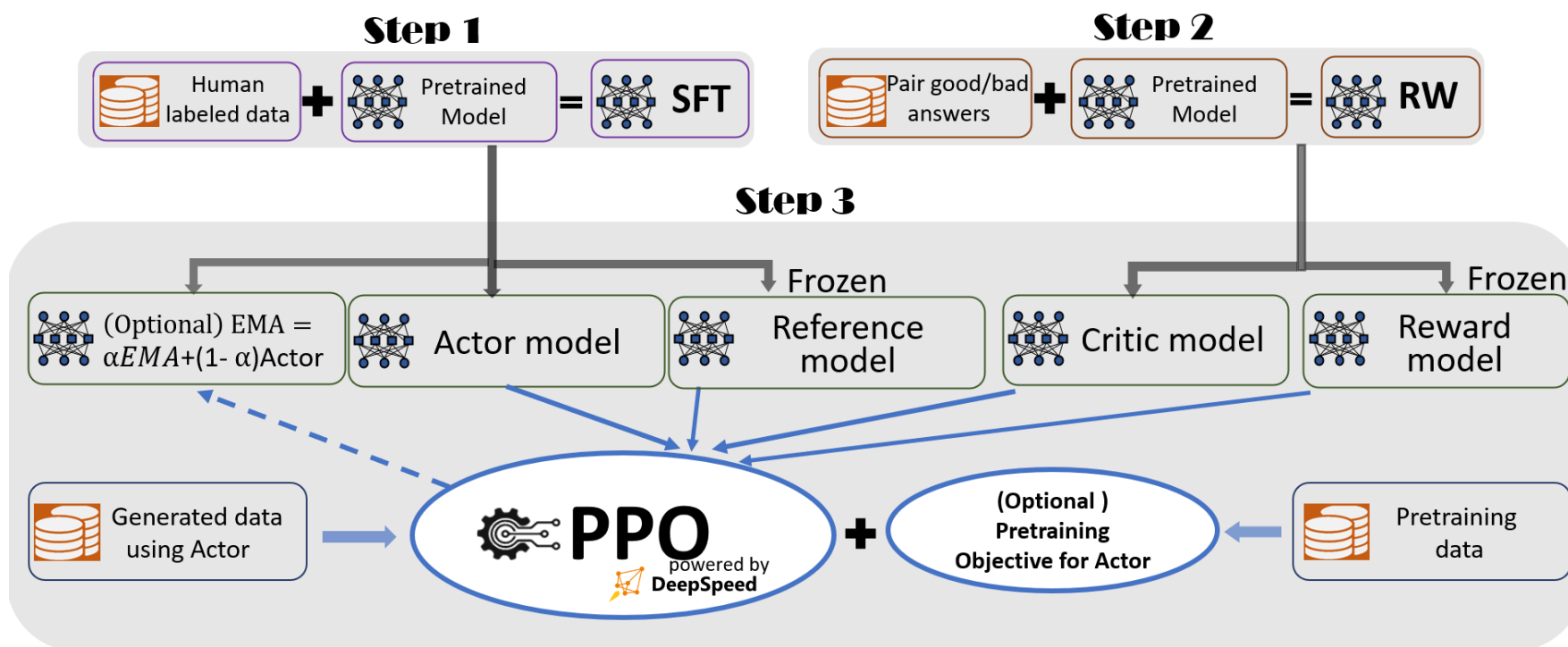
DeepSpeed-Chat[162] 是微软于2023 年4月发布的基于DeepSpeed 用于训练类ChatGPT 模型的开发工具。基于DeepSpeed-Chat 训练类ChatGPT 对话模型的步骤框架如图5.8 所示，包含以下三个步骤。

- (1) 有监督微调 (SFT)
- (2) 奖励模型微调
- (3) RLHF 训练



DeepSpeed-Chat 具有以下三大核心功能。

- (1) 易用的类ChatGPT 模型的训练和强化推理：只需要一个脚本就可以实现多个训练步骤 (2) DeepSpeed-RLHF 管道：DeepSpeed-RLHF 复现了InstructGPT[24] 论文中的训练模式； (3) DeepSpeed-RLHF 系统：将DeepSpeed 的训练能力（Training Engine）和推理能力。





DeepSpeed-Chat 代码仓库位于微软官方 GitHub 仓库 DeepSpeedExamples/applications/DeepSpeed-Chat 路径下。在进行实际应用前，需要先对官方代码有一个全局的了解。DeepSpeed-Chat 代码的结构如下所示：

```
- DeepSpeed-Chat
- inference                # 模型测试，推理
- training                 # 训练脚本
- step1_supervised_finetuning # 步骤一，有监督微调
  - training_log_output    # 训练日志输出
  - evaluation_scripts     # 监督微调模型评测
  - training_scripts       # 模型训练脚本
  - main.py                # 步骤一训练脚本
  - prompt_eval.py         # 步骤一评测脚本
  - README.md              # 说明文档
- step2_reward_model_finetuning # 步骤二，奖励模型微调
  - 省略
- step3_rlhf_finetuning     # 步骤三，RLHF训练
  - 省略
- utils                    # 模型训练与评价的相关函数库
  - data                  # 数据处理相关代码
  - model                 # 模型相关文件
  - module                # 其他组件
  - ds_utils.py           # DeepSpeed配置相关
  - utils.py              # 其他相关函数
- train.py                 # 三步骤集成训练入口
```



当需要完整微调一个模型时（包含所有步骤），可以直接运行train.py 程序。训练中主要调整如下参数。

--step 训练步骤参数，表示运行哪个步骤，可选参数为1、2、3。本节介绍的内容只使用步骤一，有监督微调。

--deployment-type 表示分布式训练模型的参数，分别为单卡single_gpu、单机多卡single_node 和 多机多卡multi_node。

--actor-model 表示要训练的模型，默认参数为训练OPT 的"1.3b"、"6.7b"、"13b"、"66b" 等各个数量的模型。

--reward-model 表示要训练的奖励模型，默认参数为OPT 的"350m" 参数量的模型。

--actor-zero-stage 表示有监督微调的DeepSpeed 分布式训练配置。

--reward-zero-stage 表示训练奖励的DeepSpeed 分布式训练配置。

--output-dir 表示训练过程和结果的输出路径。



训练一个属于自己的大语言模型，数据是非常重要的。通常，使用相关任务的数据进行优化的模型会在目标任务上表现得更好。在DeepSpeed-Chat 中使用新的数据，需要进行如下操作。

- (1) 准备数据，并把数据处理成程序能够读取的格式，如JSON、arrow。
- (2) 在数据处理代码文件training/utils/data/raw_datasets.py 和 training/utils/data/data_utils.py 中增加对新增数据的处理。
- (3) 在训练脚本中增加对新增数据的支持，并开始模型训练。



在有监督微调中，每条样本都有对应的prompt 和chosen（奖励模型微调中还有rejected）。因此，需要将新增的数据处理成如下格式（JSON）：

```
[
  {
    "prompt": " 你是谁？",
    "chosen": " 我是你的私人小助手。",
    "rejected": "",
  },
  {
    "prompt": " 讲个笑话",
    "chosen": " 为什么脚气的人不能吃香蕉？因为他们会变成香蕉脚！",
    "rejected": ""
  }
]
```



基于构建的数据，在raw_datasets.py 和data_utils.py 中增加对该数据的处理。
在raw_datasets.py 中新增如下代码，其中load(dataset_name) 为数据加载。

```
# 自定义load函数
def my_load(filepath):
    with open(filepath, 'r') as fp:
        data = json.load(fp)
    return data

# raw_datasets.py
class MyDataset(PromptRawDataset):
    def __init__(self, output_path, seed, local_rank, dataset_name):
        super().__init__(args, output_path, seed, local_rank, dataset_name)
        self.dataset_name = "MyDataset"
        # 加载数据集，其中load函数使用自定义的加载函数my_load()
        self.raw_datasets = my_load(dataset_name)

# 获取训练数据
def get_train_data(self):
    return self.raw_datasets["train"]

# 获取验证数据
def get_eval_data(self):
    return self.raw_datasets["eval"]
```

```
# 得到一个样本的prompt
def get_prompt(self, sample):
    return " Human: " + sample['prompt']

# 得到一个样本的正例回答
def get_chosen(self, sample):
    return " Assistant" + sample['chosen']

# 得到一个样本的反例回答（在这里只进行步骤一的实践介绍，因此反例样本并不会被调用）
def get_rejected(self, sample):
    return " Assistant: " + sample['rejected']

# 得到一个样本的prompt和正例回答
def get_prompt_and_chosen(self, sample):
    return " Human: " + sample['prompt'] + " Assistant: " + sample['chosen']

# 得到一个样本的prompt和反例回答
def get_prompt_and_rejected(self, sample):
    return " Human: " + sample['prompt'] + " Assistant: " + sample['rejected']
```



基于构建的数据，在raw_datasets.py 和data_utils.py 中增加对该数据的处理。
在raw_datasets.py 中新增如下代码，其中load(dataset_name) 为数据加载。

```
# data_utils.py
def get_raw_dataset(dataset_name, output_path, seed, local_rank):

    # 加入之前构建的自定义数据集
    if "MyDataset" in dataset_name:
        return raw_datasets.MyDataset(output_path, seed,
                                       local_rank, dataset_name)

    elif "Dahoas/rm-static" in dataset_name:
        return raw_datasets.DahoasRmstaticDataset(output_path, seed,
                                                  local_rank, dataset_name)

    elif "Dahoas/full-hh-rlhf" in dataset_name:
        return raw_datasets.DahoasFullhhrhlfDataset(output_path, seed,
                                                    local_rank, dataset_name)
```

数据处理完成后，读取到的数据格式如下：

```
# 原始样本
{
    "prompt": " 讲个笑话",
    "chosen": " 为什么脚气的人不能吃香蕉？因为他们会变成香蕉脚!",
    "rejected": ""
}

# 调用my_dataset.get_prompt(sample)
Human: 讲个笑话

# 调用my_dataset.get_chosen(sample)
Human: 讲个笑话 Assistant: 为什么脚气的人不能吃香蕉？因为他们会变成香蕉脚！
```



虽然DeepSpeed-Chat 内置了在各项评估上都表现良好的LLaMA-2 7B 模型，但是模型在预训练中并没有在足够的中文数据上训练，导致其中文能力并不强。当需要使用支持中文的预训练模型，或者更换其他模型时，就需要对DeepSpeed-Chat 进行相应的更改来适配其他自定义的模型。

DeepSpeed-Chat 训练中默认使用的是基于HuggingFace 格式的模型和数据，因此如果切换到Transformers 和HuggingFace 支持的模型非常简单，只需将 `model_name_or_path` 参数修改为要使用的模型即可。对于其他暂未支持的模型而言，则需要在代码层面做相应的修改。以下为、基于百川智能发布的中文大语言模型Baichuan 7B 进行自定义模型修改的具体过程。



首先进行模型结构相关的修改，在步骤一的main.py 中进行如下修改来导入相应的类：

```
# main.py
# 导入本地存储的模型相关文件
modeling_baichuan = import_module("models.Baichuan-7B.modeling_baichuan")
tokenization_baichuan = import_module("models.Baichuan-7B.tokenization_baichuan")
# 获取Baichuan模型相关的类
BaiChuanForCausalLM = getattr(modeling_baichuan, "BaiChuanForCausalLM")
BaiChuanTokenizer = getattr(tokenization_baichuan, "BaiChuanTokenizer")
```

对模型代码文件路径做相应的修改，改为本地存储模型代码的路径。然后，同样在main.py 中对对应的模型加载进行修改：

```
# main.py
# 原始代码
tokenizer = load_hf_tokenizer(args.model_name_or_path, fast_tokenizer=True)
model = create_hf_model(AutoModelForCausalLM,
                        args.model_name_or_path,
                        tokenizer,
                        ds_config,
                        disable_dropout=args.disable_dropout)

# 修改为支持Baichuan 7B的代码
tokenizer = BaiChuanTokenizer.from_pretrained(args.model_name_or_path)
model = create_hf_model(BaiChuanForCausalLM,
                        args.model_name_or_path,
                        tokenizer,
                        ds_config,
                        disable_dropout=args.disable_dropout)
```



最后，在训练脚本中将`model_name_or_path` 参数修改为Baichuan 7B 的模型路径即可开始模型的训练。训练脚本中以DeepSpeed-Chat 中的`run_llama2_7b.sh` 为模板进行修改：

```
# run_baichuan_7b.sh
#!/bin/bash
# Copyright (c) Microsoft Corporation.
# SPDX-License-Identifier: Apache-2.0

# DeepSpeed Team
OUTPUT=$1
ZERO_STAGE=$2

if [ "$OUTPUT" == "" ]; then
    OUTPUT=./output_step1_baichuan_7b
fi

if [ "$ZERO_STAGE" == "" ]; then
    ZERO_STAGE=3
fi

mkdir -p $OUTPUT

deepspeed main.py \
  --data_path <my_data>/my_dataset \ # 数据路径修改为本地的数据
  --data_split 10,0,0 \ # 由于只进行步骤一有监督微调，因此不对数据进行切分，全部用于步骤一的训练
  --model_name_or_path <my_model>/baichuan_7b \ # 模型修改为本地存储的baichuan-7b模型路径
  --per_device_train_batch_size 4 \
  --per_device_eval_batch_size 4 \
  --max_seq_len 512 \
  --learning_rate 9.65e-6 \
  --weight_decay 0. \
  --num_train_epochs 2 \
  --gradient_accumulation_steps 1 \
  --lr_scheduler_type cosine \
```

```
--num_warmup_steps 0 \
--seed 1234 \
--gradient_checkpointing \
--zero_stage $ZERO_STAGE \
--deepspeed \
--output_dir $OUTPUT \
&> $OUTPUT/training.log
```



当数据预处理和自定义模型的修改都完成后，就可以正式进行训练了。进入步骤一有监督微调的路径 `training/step1_supervised_finetuning` 下，把上述构造的训练脚本放置到 `training/step1_supervised_finetuning/training_scripts/baichuan/run_baichuan_7b.sh`，在命令行下可以运行以下代码启动训练：

```
# 在路径training/step1_supervised_finetuning下运行，示例中在一台8块NVIDIA A100机器下进行训练
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 bash training_scripts/baichuan/run_baichuan_7b.sh
```

训练进行时，会进行一次评估，计算困惑度PPL（Perplexity）。然后继续训练，在每一个epoch 结束后都会进行一次评估，PPL 也会随着训练的进行逐步下降。

模型训练完成后，可以使用DeepSpeed-Chat 根路径下的`chat.py` 进行推理。参数修改为已训练好的模型路径，具体执行方式如下：

```
# chat.py
CUDA_VISIBLE_DEVICES=0 python chat.py --path model_path
```

目录

Contents

5.1

提示学习和语境学习

5.2

高效模型微调

5.3

模型上下文窗口扩展

5.4

指令数据构建

5.5

DeepSpeed-Chat SFT实践

5.6

实践思考



指令微调的真正作用值得被进一步探索。一些工作认为经过预训练的大语言模型从训练语料中学到了大量的“知识”，但是这些模型并没有理解人类自然语言形式的命令。因此，指令微调通过构造复杂并多样的指令形式，让模型学会人类之间交流的指令。这种观点指出，指令微调所需的数据量不大，本质是让已经包含大量“知识”的语言模型学会一种输入/输出的格式。还有观点指出，预训练语言模型仍然可以通过指令微调的形式注入新的知识。研究人员可以通过构建足够数量的指令微调数据，让模型在训练过程中记住数据中的信息。基于此，一些研究人员通过在指令微调阶段引入领域数据来构建专属的领域大语言模型。



指令微调的数据量也是值得被探索的问题。LIMA[163] 证明了高质量、多样性丰富的指令数据集的指令微调可以取得以少胜多的表现。然而，模型的参数量和指令微调的数量关系仍然值得被讨论。在指令微调数据质量足够好的情况下，指令微调的数据量与模型能力的关系如何，以及更多的指令微调数据量是否会影响预训练语言模型本身的“知识”，仍是值得探讨的问题。在实践中，对于特定领域的大语言模型而言，指令数据的格式也会影响模型的性能：结构化的数据采用中文符号还是英文符号、全角符号还是半角符号，多条结构化的输出使用jsonline 形式还是jsonlist 形式，复杂任务是否构建模型的内在思考（Inner Thought），需要推理效率的场景是该将任务多步拆解还是一步端到端式解决？所以说，如何构建适合当前任务的指令数据也是一个需要在实践中仔细思考和调整的问题。



还有一个值得思考的问题是，**模型是否真正拥有数值计算的能力**。一些学者认为如今大语言模型对数字按照数码的粒度进行分词的方式难以让模型学到数值计算的能力。举个例子，当在没有借助外部工具的模型上进行简单的数值计算推理时，会发现模型通常可以计算出类似于 $199 \times 200 = 39\ 800$ 的算式。但是，当计算 199×201 时，模型就难以给出正确答案。同时，选择“÷”符号还是“/”符号作为算式中的除号也是一个值得商讨的问题。当数字缺乏充分的训练时，为了实现更好的数值计算性能，除了接入计算器API，还有一个比较有趣的解决思路是将数值计算问题和模型的代码生成能力相结合，利用代码模拟解题过程。这种做法还可以进一步与代码解释器相结合。模型接入代码解释器之后，许多问题都可以使用这种方式来解决。



如今，有很多大语言模型的派生场景都依赖于更长的上下文。对于现有的扩充模型上下文窗口的方法而言，长度外推的方法可以在整个上下文上保持较好的模型困惑度，但是额外扩充出来的窗口内的文本表现又如何呢？对于线性插值法，虽然上下文总体的模型困惑度降低了，但是模型是否能够准确地“记忆”上下文中提及的每个细节（如数字、网址等）？对于当前备受关注的模型智能体和工具学习相关的工作而言，保持上下文窗口的大小不变的同时，依然需要确保上下文中的粗细粒度的准确率（细粒度例如某事物具体的数值，粗粒度例如前文中某一部分谈论了什么话题）。对于各项研究而言，提高模型的上下文准确度是一个不可避免的问题。

本章小结

经过海量数据预训练后的语言模型虽然具备了大量的“知识”，但是由于其训练时的目标仅是进行下一个词的预测，因此不能够理解并遵循人类自然语言形式的指令。为了使模型具有理解并响应人类指令的能力，还需要使用指令数据对其进行微调。如何构造指令数据，如何高效低成本地进行指令微调训练，以及如何在语言模型基础上进一步扩大上下文等问题，是大语言模型在有监督微调阶段的核心。

本章介绍了大语言模型的提示学习与语境学习能力，在此基础上介绍高效模型微调及大语言模型上下文窗口扩展方法，最后介绍指令数据的一般格式与构建方式，以及有监督微调的代码实践。